An Advanced XVA Framework: FVA, MVA, and KVA for Central Clearing

Motivation

Post-crisis regulatory reforms, including the European Market Infrastructure Regulation (EMIR) and Basel III/IV capital requirements, have fundamentally transformed counterparty risk management in over-the-counter (OTC) derivatives markets. The mandate for central clearing of standardised derivatives and bilateral margining for non-cleared trades has created a multi-dimensional cost optimisation problem that extends far beyond traditional credit valuation adjustment (CVA) calculations.

Financial institutions now face three critical valuation adjustments that directly impact profitability and capital allocation: Funding Valuation Adjustment (FVA), which captures the treasury funding costs associated with uncollateralised exposures; Margin Valuation Adjustment (MVA), which prices the cost of initial margin over the trade lifecycle; and Capital Valuation Adjustment (KVA), which quantifies the regulatory capital costs under Basel III/IV frameworks.

For centrally cleared portfolios, the Margin Period of Risk (MPoR) model becomes critical for Close-Out Amount (COA) simulation, enabling analysis of Central Counterparty (CCP) default waterfall mechanisms. This project develops a comprehensive XVA engine that incorporates funding spread dynamics, regulatory capital requirements, and CCP margin algorithms to provide a complete picture of the economic cost of derivative trading in the post-crisis regulatory environment.

By combining rigorous mathematical foundations with an interactive visualisation platform, this tool enables quantitative analysts, risk managers, and treasury teams to understand the full cost structure of derivative portfolios, optimise capital allocation, and make informed strategic decisions in an increasingly regulated marketplace.

Theory

This project implements a comprehensive XVA framework for centrally cleared portfolios, incorporating funding costs, margin requirements, and regulatory capital charges.

Funding Valuation Adjustment (FVA)

Definition (Funding Spread)

The funding spread $s_F(t)$ represents the difference between the bank's funding rate and the risk-free rate at time t. For a derivative portfolio with exposure $E(t)$, the FVA captures the cost of funding uncollateralised exposures.

The Funding Valuation Adjustment quantifies the cost of funding uncollateralised derivative exposures. For a portfolio with exposure profile $\{E(t)\}_{t\geq 0}$, the FVA is given by:

$$\text{FVA} = \int_0^T \mathbb{E}^{\mathbb{Q}}\left[s_F(t)E^+(t)e^{-\int_0^t r(u)\,\mathrm{d}u}\right]\,\mathrm{d}t,$$

where $E^+(t) = \max(E(t), 0)$ represents the positive exposure (funding requirement), $s_F(t)$ is the funding spread, $r(t)$ is the risk-free rate, and $\mathbb{E}^{\mathbb{Q}}$ denotes expectation under the risk-neutral measure.

In discrete time, this becomes:

$$\text{FVA} = \sum_{i=0}^{N-1} \mathbb{E}^{\mathbb{Q}} \left[s_F(t_i) E^+(t_i) D(t_i) \right] \Delta t_i,$$

where $D(t_i) = e^{-\int_0^{t_i} r(u)\,\mathrm{d}u}$ is the discount factor and Δt_i is the time step.

Margin Valuation Adjustment (MVA)

Theorem (Initial Margin Cost)

For a portfolio subject to initial margin requirements $\{IM(t)\}_{t \geq 0}$, the MVA represents the present value of funding costs associated with posting initial margin over the trade lifecycle:

$$\text{MVA} = \int_0^T \mathbb{E}^{\mathbb{Q}} \left[s_F(t) IM(t) e^{-\int_0^t r(u)\,\mathrm{d}u} \right] \mathrm{d}t.$$

Initial margin is calculated using standardised methodologies such as ISDA SIMM (Standard Initial Margin Model), which aggregates risk sensitivities across delta, vega, and curvature risk factors:

$$IM(t) = \sqrt{\sum_k \left(\sum_i \Delta_i \cdot RW_i \cdot \rho_{i,k} \right)^2 + \sum_k \left(\sum_i \text{Vega}_i \cdot RW_i^{\text{vega}} \right)^2 + \text{Curvature}},$$

where Δ_i are delta sensitivities, Vega_i are vega sensitivities, RW_i are risk weights, and $\rho_{i,k}$ are correlation factors between risk factors i and k.

For CCP-cleared trades, initial margin follows CCP-specific algorithms (e.g., CME SPAN, LCH's margin methodology), which typically incorporate:

- Portfolio-level netting benefits
- Stress scenario analysis
- Volatility and correlation assumptions
- Concentration risk adjustments

Capital Valuation Adjustment (KVA)

Definition (Regulatory Capital Cost)

Under Basel III/IV, banks must hold regulatory capital against CVA risk. The KVA quantifies the cost of this capital requirement, incorporating the cost of capital (CoC) and the capital multiplier.

The Capital Valuation Adjustment captures the cost of regulatory capital requirements. For CVA capital under Basel III, the capital charge is:

$$K_{\text{CVA}} = \alpha \cdot \sqrt{\sum_i \sum_j \rho_{i,j} \cdot RW_i \cdot EAD_i \cdot RW_j \cdot EAD_j},$$

where α is the CVA multiplier (typically 1.25), RW_i is the risk weight for counterparty i (0.50 for OTC derivatives), EAD_i is the exposure at default, and $\rho_{i,j}$ is the correlation between counterparties i and j.

The KVA is then:

$$\text{KVA} = \text{CoC} \cdot \int_0^T \mathbb{E}^{\mathbb{Q}} \left[K_{\text{CVA}}(t) e^{-\int_0^t r(u)\,\mathrm{d}u} \right] \mathrm{d}t,$$

where CoC is the cost of capital (e.g., 10% for CET1 target of 12%).

For a portfolio with multiple counterparties, the capital calculation incorporates diversification benefits through correlation matrices, whilst maintaining minimum capital floors as specified by regulatory frameworks.

Margin Period of Risk and Close-Out Amount

Theorem (Close-Out Amount Simulation)

The Close-Out Amount (COA) represents the replacement cost of a portfolio upon counterparty default. Under the MPoR framework, the COA is simulated over the margin period of risk τ_{MPoR}:

$$\text{COA}(t) = \max\left(V(t + \tau_{\text{MPoR}}) - IM(t) - VM(t), 0\right),$$

where $V(t)$ is the portfolio value, $IM(t)$ is initial margin, and $VM(t)$ is variation margin at time t.

The Margin Period of Risk represents the time between the last margin exchange and the point at which the portfolio can be replaced or hedged. For CCP-cleared trades, the MPoR is typically shorter (e.g., 2-5 days) compared to bilateral trades (10-20 days), reflecting the operational efficiency of central clearing.

The COA simulation requires:

- Monte Carlo simulation of market risk factors over τ_{MPoR}
- Portfolio revaluation at each simulation path
- Incorporation of margin call frequency and settlement delays
- Stress scenario analysis for extreme market movements

CCP Default Waterfall Analysis

For centrally cleared portfolios, understanding the CCP default waterfall is essential for risk management. The waterfall typically follows this hierarchy:

1. **Defaulting member's margin**: Initial margin and variation margin posted by the defaulting member
2. **Defaulting member's contribution to default fund**: Pre-funded contributions
3. **CCP's own capital**: Equity and retained earnings
4. **Non-defaulting members' default fund contributions**: Mutualised losses
5. **Assessment rights**: Additional contributions from clearing members

The probability of loss beyond a member's own margin and default fund contribution can be modelled using extreme value theory and stress testing scenarios.

Portfolio-Level XVA Aggregation

For a portfolio of N trades, the total XVA is:

$$\text{Total XVA} = \sum_{i=1}^{N} \text{FVA}_i + \sum_{i=1}^{N} \text{MVA}_i + \text{KVA}_{\text{portfolio}},$$

where portfolio-level KVA incorporates netting benefits and diversification effects. The portfolio KVA is typically less than the sum of individual trade KVAs due to:

- Netting agreements reducing net exposure
- Correlation effects in capital calculations
- Portfolio-level risk aggregation benefits

Funding Spread Dynamics

Funding spreads exhibit stochastic behaviour driven by:

- Credit quality of the institution
- Market liquidity conditions
- Cross-currency basis effects
- Regulatory capital constraints

A common modelling approach uses a mean-reverting process:

$$\mathrm{d}s_F(t) = \kappa(\theta - s_F(t))\,\mathrm{d}t + \sigma_F\sqrt{s_F(t)}\,\mathrm{d}W_F(t),$$

where κ is the mean reversion speed, θ is the long-term mean, and σ_F is the volatility of the funding spread.

Data Requirements

- **Funding spreads**: 3-month USD Libor-OIS basis, treasury cross-currency basis spreads
- **Initial margin schedules**: ISDA SIMM risk weights for delta, vega, and curvature
- **Regulatory parameters**: CVA multiplier ($\alpha = 1.25$), risk weights ($RW = 0.50$ for OTC derivatives)
- **Capital ratios**: CET1 target (e.g., 12%), Cost of Capital (CoC = 10%)
- **CCP margin rules**: CME/LCH margin algorithm parameters, MPoR assumptions
- **Portfolio data**: Trade details (notional, maturity, counterparty), market data (rates, volatilities, correlations)
- **Counterparty data**: CDS spreads (80-200 bps for major dealers), credit ratings

The tool includes synthetic data generation capabilities for demonstration purposes, creating realistic portfolios and market data scenarios.

Code

The following code excerpts illustrate production-quality implementations using object-oriented design principles.

Python: Object-Oriented XVA Engine

```python
import numpy as np
import pandas as pd
from typing import Dict, List, Optional, Tuple
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from scipy.stats import norm
import warnings


@dataclass
class Trade:
    """Represents a single derivative trade."""
    trade_id: str
    counterparty: str
    notional: float
    maturity: float  # Years
    currency: str
    trade_type: str  # 'IRS', 'CCS', 'FX', etc.
    fixed_rate: Optional[float] = None
    float_index: Optional[str] = None
    start_date: float = 0.0  # Years from today

    def __post_init__(self):
        """Validate trade parameters."""
        if self.notional <= 0:
            raise ValueError("Notional must be positive")
        if self.maturity <= 0:
            raise ValueError("Maturity must be positive")


@dataclass
class MarketData:
    """Market data container for XVA calculations."""
    risk_free_rate: float
    funding_spread: float  # Funding spread over risk-free rate
    cds_spread: Dict[str, float]  # Counterparty CDS spreads (as
       decimals)
    volatility: Dict[str, float]  # Volatility by currency/asset
    correlations: np.ndarray  # Correlation matrix
    initial_margin_risk_weight: float = 0.02  # ISDA SIMM risk weight
    mpor: float = 5.0 / 252.0  # Margin Period of Risk (5 days)

    def __post_init__(self):
        """Validate market data."""
        if self.risk_free_rate < 0:
            warnings.warn("Negative risk-free rate")
        if self.funding_spread < 0:
            warnings.warn("Negative funding spread")


```

```python
class XVACalculator(ABC):
    """Abstract base class for XVA calculations."""

    @abstractmethod
    def calculate(self, portfolio: List[Trade],
                  market_data: MarketData,
                  time_grid: Optional[np.ndarray] = None) -> Dict:
        """
        Calculate XVA adjustment.

        Args:
            portfolio: List of Trade objects
            market_data: MarketData object
            time_grid: Optional time grid for calculations

        Returns:
            Dictionary containing XVA results and intermediate
               calculations
        """
        pass


class ExposureSimulator:
    """Simulates portfolio exposure paths using Monte Carlo."""

    def __init__(self, num_paths: int = 10000, num_steps: int = 252):
        """
        Initialise exposure simulator.

        Args:
            num_paths: Number of Monte Carlo paths
            num_steps: Number of time steps per year
        """
        self.num_paths = num_paths
        self.num_steps = num_steps

    def simulate_exposure(self,
                          portfolio: List[Trade],
                          market_data: MarketData,
                          time_grid: np.ndarray,
                          random_seed: Optional[int] = None) -> np.
                             ndarray:
        """
        Simulate exposure paths using Monte Carlo.

        Args:
            portfolio: List of Trade objects
            market_data: MarketData object
            time_grid: Time grid for simulation
            random_seed: Random seed for reproducibility

        Returns:
            Array of shape (num_paths, len(time_grid)) with exposure
               paths
        """
        if random_seed is not None:
            np.random.seed(random_seed)

```

```
        exposures = np.zeros((self.num_paths, len(time_grid)))

        # Get portfolio parameters
        max_maturity = max(trade.maturity for trade in portfolio)
        currencies = list(set(trade.currency for trade in portfolio))

        # Generate correlated random shocks
        num_factors = len(currencies)
        if num_factors == 0:
            return exposures

        # Simplified: use single factor model
        for path in range(self.num_paths):
            # Generate random shocks
            dt = time_grid[1] - time_grid[0] if len(time_grid) > 1 else
                0.01
            shocks = np.random.normal(0, np.sqrt(dt), len(time_grid))

            for i, t in enumerate(time_grid):
                exposure = 0.0

                for trade in portfolio:
                    if trade.start_date <= t < trade.maturity:
                        # Simplified trade valuation
                        time_to_maturity = trade.maturity - t

                        if trade.trade_type == 'IRS':
                            # Interest rate swap valuation
                            vol = market_data.volatility.get(trade.
                                currency, 0.15)
                            # Simplified: value depends on rate
                                movement
                            rate_shock = shocks[i] * vol
                            value = (trade.notional *
                                    (trade.fixed_rate - market_data.
                                        risk_free_rate - rate_shock) *
                                    time_to_maturity)
                            exposure += value

                        elif trade.trade_type == 'CCS':
                            # Cross-currency swap
                            vol = market_data.volatility.get(trade.
                                currency, 0.15)
                            fx_shock = shocks[i] * vol
                            value = trade.notional * fx_shock *
                                time_to_maturity
                            exposure += value

                        else:
                            # Generic trade
                            vol = market_data.volatility.get(trade.
                                currency, 0.15)
                            value = trade.notional * shocks[i] * vol *
                                np.sqrt(time_to_maturity)
                            exposure += value

                exposures[path, i] = exposure

```

```
        return exposures

    def calculate_expected_exposure(self, exposures: np.ndarray) -> np.ndarray:
        """Calculate expected exposure profile."""
        return np.mean(exposures, axis=0)

    def calculate_positive_exposure(self, exposures: np.ndarray) -> np.ndarray:
        """Calculate expected positive exposure (EPE)."""
        positive_exposures = np.maximum(exposures, 0)
        return np.mean(positive_exposures, axis=0)

    def calculate_effective_expected_exposure(self, epe: np.ndarray) -> np.ndarray:
        """Calculate Effective Expected Exposure (running maximum of EPE)."""
        eee = np.zeros_like(epe)
        current_max = 0.0
        for i, val in enumerate(epe):
            current_max = max(current_max, val)
            eee[i] = current_max
        return eee


class FVACalculator(XVACalculator):
    """Funding Valuation Adjustment calculator."""

    def __init__(self, num_paths: int = 10000, num_steps: int = 252):
        """
        Initialise FVA calculator.

        Args:
            num_paths: Number of Monte Carlo paths
            num_steps: Number of time steps per year
        """
        self.num_paths = num_paths
        self.num_steps = num_steps
        self.exposure_simulator = ExposureSimulator(num_paths, num_steps)

    def calculate(self, portfolio: List[Trade],
                  market_data: MarketData,
                  time_grid: Optional[np.ndarray] = None) -> Dict:
        """
        Calculate FVA.

        Returns:
            Dictionary with FVA value and intermediate calculations
        """
        if not portfolio:
            return {'FVA': 0.0, 'exposures': None, 'epe': None}

        # Create time grid
        max_maturity = max(trade.maturity for trade in portfolio)
        if time_grid is None:
            num_points = int(max_maturity * self.num_steps) + 1
            time_grid = np.linspace(0, max_maturity, num_points)
```

```
208
209         dt = time_grid[1] - time_grid[0] if len(time_grid) > 1 else
                0.01
210
211         # Simulate exposures
212         exposures = self.exposure_simulator.simulate_exposure(
213             portfolio, market_data, time_grid
214         )
215
216         # Calculate Expected Positive Exposure (EPE)
217         epe = self.exposure_simulator.calculate_positive_exposure(
                exposures)
218
219         # Calculate FVA
220         fva = 0.0
221         for i, t in enumerate(time_grid[:-1]):
222             discount_factor = np.exp(-market_data.risk_free_rate * t)
223             fva += market_data.funding_spread * epe[i] *
                        discount_factor * dt
224
225         return {
226             'FVA': fva,
227             'exposures': exposures,
228             'epe': epe,
229             'time_grid': time_grid
230         }
231
232
233 class MVACalculator(XVACalculator):
234     """Margin Valuation Adjustment calculator."""
235
236     def __init__(self, mpor: float = 5.0 / 252.0):
237         """
238         Initialise MVA calculator.
239
240         Args:
241             mpor: Margin Period of Risk in years (default 5 days)
242         """
243         self.mpor = mpor
244
245     def calculate_initial_margin_simm(self,
246                                       portfolio: List[Trade],
247                                       market_data: MarketData,
248                                       t: float) -> float:
249         """
250         Calculate initial margin using simplified ISDA SIMM methodology
                .
251
252         Args:
253             portfolio: List of Trade objects
254             market_data: MarketData object
255             t: Time point
256
257         Returns:
258             Initial margin amount
259         """
260         # Simplified SIMM calculation
```

```
        # In practice, this would involve full risk sensitivity aggregation

        total_delta_risk = 0.0
        total_vega_risk = 0.0

        for trade in portfolio:
            if trade.start_date <= t < trade.maturity:
                time_to_maturity = trade.maturity - t

                # Delta risk (simplified)
                vol = market_data.volatility.get(trade.currency, 0.15)
                delta_risk = trade.notional * vol * np.sqrt(time_to_maturity)
                total_delta_risk += delta_risk

                # Vega risk (simplified)
                vega_risk = trade.notional * vol * time_to_maturity
                total_vega_risk += vega_risk

        # Aggregate risks with correlation (simplified)
        # In practice, use full SIMM correlation matrix
        correlation = 0.5  # Simplified correlation
        aggregated_risk = np.sqrt(
            total_delta_risk**2 +
            total_vega_risk**2 +
            2 * correlation * total_delta_risk * total_vega_risk
        )

        # Apply risk weight
        im = aggregated_risk * market_data.initial_margin_risk_weight

        # Add concentration and curvature adjustments (simplified)
        im *= 1.1  # 10% adjustment

        return max(im, 0.0)

    def calculate(self, portfolio: List[Trade],
                  market_data: MarketData,
                  time_grid: Optional[np.ndarray] = None) -> Dict:
        """
        Calculate MVA.

        Returns:
            Dictionary with MVA value and initial margin profile
        """
        if not portfolio:
            return {'MVA': 0.0, 'im_profile': None}

        # Create time grid
        max_maturity = max(trade.maturity for trade in portfolio)
        if time_grid is None:
            num_points = int(max_maturity * 252) + 1
            time_grid = np.linspace(0, max_maturity, num_points)

        dt = time_grid[1] - time_grid[0] if len(time_grid) > 1 else 0.01

```

```
        # Calculate initial margin over time
        im_profile = np.zeros(len(time_grid))
        for i, t in enumerate(time_grid):
            im_profile[i] = self.calculate_initial_margin_simm(
                portfolio, market_data, t
            )

        # Calculate MVA
        mva = 0.0
        for i, t in enumerate(time_grid):
            discount_factor = np.exp(-market_data.risk_free_rate * t)
            mva += market_data.funding_spread * im_profile[i] *
               discount_factor * dt

        return {
            'MVA': mva,
            'im_profile': im_profile,
            'time_grid': time_grid
        }


class KVACalculator(XVACalculator):
    """Capital Valuation Adjustment calculator."""

    def __init__(self, cva_multiplier: float = 1.25,
                 risk_weight: float = 0.50,
                 cost_of_capital: float = 0.10):
        """
        Initialise KVA calculator.

        Args:
            cva_multiplier: CVA multiplier (Basel III: 1.25)
            risk_weight: Risk weight for OTC derivatives (0.50)
            cost_of_capital: Cost of capital (e.g., 10%)
        """
        self.cva_multiplier = cva_multiplier
        self.risk_weight = risk_weight
        self.cost_of_capital = cost_of_capital

    def calculate_cva_capital(self,
                              portfolio: List[Trade],
                              market_data: MarketData,
                              time_grid: np.ndarray,
                              exposures: Optional[np.ndarray] = None)
                                 -> np.ndarray:
        """
        Calculate CVA capital requirement over time.

        Args:
            portfolio: List of Trade objects
            market_data: MarketData object
            time_grid: Time grid
            exposures: Optional exposure array (if None, uses
               simplified calculation)

        Returns:
            Array of capital requirements over time
        """
```

```
        capital = np.zeros(len(time_grid))

        # Group trades by counterparty
        counterparties = list(set(trade.counterparty for trade in
            portfolio))

        for i, t in enumerate(time_grid):
            # Calculate EAD (Exposure at Default) by counterparty
            ead_by_cp = {}

            if exposures is not None:
                # Use simulated exposures
                epe = np.mean(np.maximum(exposures[:, i], 0))
                # Distribute EPE across counterparties (simplified)
                for cp in counterparties:
                    cp_trades = [trade for trade in portfolio if trade.
                        counterparty == cp
                                and trade.start_date <= t < trade.
                                    maturity]
                    total_notional = sum(
                        trade.notional for trade in portfolio if trade.
                            start_date <= t < trade.maturity
                    )
                    cp_weight = (sum(trade.notional for trade in
                        cp_trades) / total_notional
                                 if total_notional > 0 else 0)
                    ead_by_cp[cp] = epe * cp_weight
            else:
                # Simplified EAD calculation
                for trade in portfolio:
                    if trade.start_date <= t < trade.maturity:
                        cp = trade.counterparty
                        if cp not in ead_by_cp:
                            ead_by_cp[cp] = 0.0
                        # Simplified: EAD as percentage of notional
                        ead_by_cp[cp] += abs(trade.notional * 0.1)

            # Calculate capital with correlation
            # Basel III formula: K_CVA = alpha * sqrt(sum_i sum_j
                rho_ij * RW_i * EAD_i * RW_j * EAD_j)
            total_capital_squared = 0.0

            for cp1 in counterparties:
                for cp2 in counterparties:
                    ead1 = ead_by_cp.get(cp1, 0.0)
                    ead2 = ead_by_cp.get(cp2, 0.0)

                    # Correlation: 1.0 for same counterparty, 0.3-0.5
                        for different
                    if cp1 == cp2:
                        correlation = 1.0
                    else:
                        correlation = 0.3

                    contribution = (self.risk_weight * ead1 *
                                    self.risk_weight * ead2 * correlation
                                        )
                    total_capital_squared += contribution
```

```
            capital[i] = self.cva_multiplier * np.sqrt(max(
                total_capital_squared, 0))

        return capital

    def calculate(self, portfolio: List[Trade],
                  market_data: MarketData,
                  time_grid: Optional[np.ndarray] = None,
                  exposures: Optional[np.ndarray] = None) -> Dict:
        """
        Calculate KVA.

        Args:
            portfolio: List of Trade objects
            market_data: MarketData object
            time_grid: Optional time grid
            exposures: Optional exposure array from FVA calculation

        Returns:
            Dictionary with KVA value and capital profile
        """
        if not portfolio:
            return {'KVA': 0.0, 'capital_profile': None}

        # Create time grid
        max_maturity = max(trade.maturity for trade in portfolio)
        if time_grid is None:
            num_points = int(max_maturity * 252) + 1
            time_grid = np.linspace(0, max_maturity, num_points)

        dt = time_grid[1] - time_grid[0] if len(time_grid) > 1 else
            0.01

        # Calculate capital over time
        capital = self.calculate_cva_capital(
            portfolio, market_data, time_grid, exposures
        )

        # Calculate KVA
        kva = 0.0
        for i, t in enumerate(time_grid):
            discount_factor = np.exp(-market_data.risk_free_rate * t)
            kva += self.cost_of_capital * capital[i] * discount_factor
                * dt

        return {
            'KVA': kva,
            'capital_profile': capital,
            'time_grid': time_grid
        }


class COASimulator:
    """Close-Out Amount simulator for MPoR analysis."""

    def __init__(self, mpor: float = 5.0 / 252.0, num_paths: int =
        10000):
```

```
        """
        Initialise COA simulator.

        Args:
            mpor: Margin Period of Risk in years
            num_paths: Number of simulation paths
        """
        self.mpor = mpor
        self.num_paths = num_paths

    def simulate_coa(self,
                     portfolio: List[Trade],
                     market_data: MarketData,
                     t: float,
                     initial_margin: float,
                     variation_margin: float = 0.0) -> np.ndarray:
        """
        Simulate Close-Out Amount at time t.

        Args:
            portfolio: List of Trade objects
            market_data: MarketData object
            t: Current time
            initial_margin: Initial margin posted
            variation_margin: Variation margin at time t

        Returns:
            Array of COA values across simulation paths
        """
        # Simulate portfolio value at t + MPoR
        coa_values = np.zeros(self.num_paths)

        for path in range(self.num_paths):
            # Simulate portfolio value change over MPoR
            portfolio_value_change = 0.0

            for trade in portfolio:
                if trade.start_date <= t < trade.maturity:
                    vol = market_data.volatility.get(trade.currency,
                        0.15)
                    shock = np.random.normal(0, vol * np.sqrt(self.mpor
                        ))

                    if trade.trade_type == 'IRS':
                        value_change = trade.notional * shock * (trade.
                            maturity - t)
                    else:
                        value_change = trade.notional * shock * np.sqrt
                            (trade.maturity - t)

                    portfolio_value_change += value_change

            # COA = max(portfolio_value_change - IM - VM, 0)
            coa_values[path] = max(
                portfolio_value_change - initial_margin -
                    variation_margin, 0
            )

```

```
        return coa_values


class XVAEngine:
    """Main XVA calculation engine."""

    def __init__(self,
                 num_paths: int = 10000,
                 num_steps: int = 252,
                 cva_multiplier: float = 1.25,
                 risk_weight: float = 0.50,
                 cost_of_capital: float = 0.10):
        """
        Initialise XVA engine.

        Args:
            num_paths: Number of Monte Carlo paths
            num_steps: Number of time steps per year
            cva_multiplier: CVA multiplier for capital calculation
            risk_weight: Risk weight for capital calculation
            cost_of_capital: Cost of capital
        """
        self.fva_calculator = FVACalculator(num_paths, num_steps)
        self.mva_calculator = MVACalculator()
        self.kva_calculator = KVACalculator(
            cva_multiplier, risk_weight, cost_of_capital
        )
        self.coa_simulator = COASimulator()

    def calculate_all_xva(self,
                          portfolio: List[Trade],
                          market_data: MarketData) -> Dict:
        """
        Calculate all XVA components.

        Args:
            portfolio: List of Trade objects
            market_data: MarketData object

        Returns:
            Dictionary containing all XVA results
        """
        if not portfolio:
            return {
                'FVA': 0.0,
                'MVA': 0.0,
                'KVA': 0.0,
                'Total XVA': 0.0
            }

        # Create common time grid using configured num_steps
        max_maturity = max(trade.maturity for trade in portfolio)
        num_points = int(max_maturity * self.fva_calculator.num_steps)
            + 1
        time_grid = np.linspace(0, max_maturity, num_points)

        # Calculate FVA (includes exposure simulation)
```

```
        fva_results = self.fva_calculator.calculate(portfolio,
            market_data, time_grid)

        # Calculate MVA
        mva_results = self.mva_calculator.calculate(portfolio,
            market_data, time_grid)

        # Calculate KVA (reuse exposures from FVA)
        kva_results = self.kva_calculator.calculate(
            portfolio, market_data, time_grid, fva_results['exposures']
        )

        # Aggregate results
        total_xva = (fva_results['FVA'] +
                     mva_results['MVA'] +
                     kva_results['KVA'])

        return {
            'FVA': fva_results['FVA'],
            'MVA': mva_results['MVA'],
            'KVA': kva_results['KVA'],
            'Total XVA': total_xva,
            'FVA_details': fva_results,
            'MVA_details': mva_results,
            'KVA_details': kva_results,
            'time_grid': time_grid
        }


def generate_synthetic_portfolio(num_trades: int = 50,
                                 total_notional: float = 250e6,
                                 counterparties: Optional[List[str]] =
                                     None,
                                 random_seed: Optional[int] = 42) ->
                                     List[Trade]:
    """
    Generate synthetic derivative portfolio for testing.

    Args:
        num_trades: Number of trades to generate
        total_notional: Total notional amount
        counterparties: List of counterparty names (default: CP1-CP8)
        random_seed: Random seed for reproducibility

    Returns:
        List of Trade objects
    """
    if random_seed is not None:
        np.random.seed(random_seed)

    if counterparties is None:
        counterparties = [f'CP{i+1}' for i in range(8)]

    trades = []
    currencies = ['USD', 'EUR', 'GBP']
    trade_types = ['IRS', 'CCS', 'FX']

    # Distribute notional across trades
```

```
    notional_per_trade = total_notional / num_trades

    for i in range(num_trades):
        # Random trade parameters
        notional = notional_per_trade * np.random.uniform(0.5, 1.5)
        maturity = np.random.uniform(1.0, 30.0)  # 1-30 years
        currency = np.random.choice(currencies)
        trade_type = np.random.choice(trade_types)
        counterparty = np.random.choice(counterparties)

        # Fixed rate for IRS
        fixed_rate = None
        if trade_type == 'IRS':
            fixed_rate = np.random.uniform(0.01, 0.05)  # 1-5%

        trade = Trade(
            trade_id=f'Trade_{i+1:04d}',
            counterparty=counterparty,
            notional=notional,
            maturity=maturity,
            currency=currency,
            trade_type=trade_type,
            fixed_rate=fixed_rate,
            start_date=0.0
        )
        trades.append(trade)

    return trades
```

This object-oriented design ensures modularity and extensibility, allowing for easy integration of additional XVA components or alternative calculation methodologies.

Python: Streamlit Visual XVA Dashboard

```
import streamlit as st
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
from xva_engine import (
    XVAEngine, Trade, MarketData,
    generate_synthetic_portfolio,
    ExposureSimulator
)

# Page configuration
st.set_page_config(
    page_title="XVA Framework Dashboard",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom CSS
st.markdown("""
    <style>
    .main-header {
        font-size: 2.5rem;
```

```
        font-weight: bold;
        color: #6865F2;
        text-align: center;
        margin-bottom: 1rem;
    }
    .metric-card {
        background-color: #f0f2f6;
        padding: 1rem;
        border-radius: 0.5rem;
        border-left: 4px solid #6865F2;
    }
    .stProgress > div > div > div {
        background-color: #6865F2;
    }
    </style>
""", unsafe_allow_html=True)

# Main header
st.markdown(
    '<h1 class="main-header">XVA Framework for Central Clearing</h1>',
    unsafe_allow_html=True
)

st.markdown("""
    <div style='text-align: center; color: #666; margin-bottom: 2rem;'>
        Funding, Margin, and Capital Valuation Adjustments (FVA, MVA, KVA)
        for centrally cleared derivative portfolios under Basel III/IV and EMIR.
    </div>
""", unsafe_allow_html=True)

st.markdown("---")

# Initialize session state
if 'portfolio' not in st.session_state:
    st.session_state.portfolio = None
if 'market_data' not in st.session_state:
    st.session_state.market_data = None
if 'xva_results' not in st.session_state:
    st.session_state.xva_results = None

# Sidebar for inputs
with st.sidebar:
    st.header("Portfolio Configuration")

    num_trades = st.slider(
        "Number of Trades",
        min_value=10,
        max_value=100,
        value=50,
        step=10,
        help="Number of trades in the portfolio"
    )

    total_notional = st.number_input(
        "Total Notional (Million)",
        min_value=100.0,
```

```
        max_value=1000.0,
        value=250.0,
        step=50.0,
        format="%.0f"
    )

    if st.button("Generate Portfolio", type="primary"):
        with st.spinner("Generating synthetic portfolio..."):
            portfolio = generate_synthetic_portfolio(
                num_trades, total_notional * 1e6, random_seed=42
            )
            st.session_state.portfolio = portfolio
            st.success(f"Generated {len(portfolio)} trades")

    st.markdown("---")
    st.header("Market Parameters")

    risk_free_rate = st.slider(
        "Risk-Free Rate (%)",
        min_value=0.0,
        max_value=5.0,
        value=2.5,
        step=0.1,
        help="Risk-free interest rate"
    ) / 100

    funding_spread = st.slider(
        "Funding Spread (bps)",
        min_value=10,
        max_value=200,
        value=50,
        step=5,
        help="Funding spread over risk-free rate"
    ) / 10000

    avg_cds_spread = st.slider(
        "Average CDS Spread (bps)",
        min_value=80,
        max_value=200,
        value=120,
        step=10,
        help="Average counterparty CDS spread"
    ) / 10000

    st.markdown("---")
    st.header("Regulatory Parameters")

    cva_multiplier = st.slider(
        "CVA Multiplier (  )",
        min_value=1.0,
        max_value=2.0,
        value=1.25,
        step=0.05,
        help="Basel III CVA multiplier"
    )

    risk_weight = st.slider(
        "Risk Weight (RW)",
```

```
        min_value=0.1,
        max_value=1.0,
        value=0.50,
        step=0.05,
        help="Risk weight for OTC derivatives"
    )

    cost_of_capital = st.slider(
        "Cost of Capital (%)",
        min_value=5.0,
        max_value=15.0,
        value=10.0,
        step=0.5,
        help="Cost of capital for KVA calculation"
    ) / 100

    st.markdown("---")
    st.header("Simulation Settings")

    num_paths = st.slider(
        "Monte Carlo Paths",
        min_value=50,
        max_value=500,
        value=200,
        step=50,
        help="Number of simulation paths (more = slower but more
            accurate). Start with 200 for faster results."
    )

    time_steps_per_year = st.slider(
        "Time Steps per Year",
        min_value=12,
        max_value=52,
        value=24,
        step=12,
        help="Number of time steps per year (fewer = faster). 24 = bi-
            weekly, 52 = weekly"
    )

# Main content area
if st.session_state.portfolio is None:
    st.info("Please generate a portfolio using the sidebar controls.")
    st.markdown("""
    ### Getting Started:
    1. Configure portfolio size and notional
    2. Click "Generate Portfolio" to create synthetic trades
    3. Adjust market and regulatory parameters
    4. Explore XVA calculations and visualisations
    """)
else:
    portfolio = st.session_state.portfolio

    # Create market data
    counterparties = list(set(t.counterparty for t in portfolio))
    cds_spreads = {cp: avg_cds_spread * np.random.uniform(0.8, 1.2)
                   for cp in counterparties}

    market_data = MarketData(
```

```
        risk_free_rate=risk_free_rate,
        funding_spread=funding_spread,
        cds_spread=cds_spreads,
        volatility={'USD': 0.12, 'EUR': 0.15, 'GBP': 0.13},
        correlations=np.array([[1.0, 0.5, 0.4], [0.5, 1.0, 0.6], [0.4,
            0.6, 1.0]])
    )

    st.session_state.market_data = market_data

    # Calculate XVA
    with st.spinner("Calculating XVA components..."):
        engine = XVAEngine(
            num_paths=num_paths,
            num_steps=time_steps_per_year,  # Use configurable time
                steps
            cva_multiplier=cva_multiplier,
            risk_weight=risk_weight,
            cost_of_capital=cost_of_capital
        )

        xva_results = engine.calculate_all_xva(portfolio, market_data)
        st.session_state.xva_results = xva_results

    # Key metrics
    st.subheader("XVA Summary")

    col1, col2, col3, col4 = st.columns(4)

    with col1:
        fva_pct = xva_results['FVA']/xva_results['Total XVA']*100
        st.metric(
            label="FVA",
            value=f"  {xva_results['FVA']:,.2f}",
            delta=f"{fva_pct:.1f}% of Total XVA"
        )

    with col2:
        mva_pct = xva_results['MVA']/xva_results['Total XVA']*100
        st.metric(
            label="MVA",
            value=f"  {xva_results['MVA']:,.2f}",
            delta=f"{mva_pct:.1f}% of Total XVA"
        )

    with col3:
        kva_pct = xva_results['KVA']/xva_results['Total XVA']*100
        st.metric(
            label="KVA",
            value=f"  {xva_results['KVA']:,.2f}",
            delta=f"{kva_pct:.1f}% of Total XVA"
        )

    with col4:
        total_notional_value = total_notional * 1e6
        xva_pct_notional = (xva_results['Total XVA']/
            total_notional_value)*100
        st.metric(
```

```
            label="Total XVA",
            value=f"  {xva_results['Total XVA']:,.2f}",
            delta=f"{xva_pct_notional:.3f}% of notional"
        )

    st.markdown("---")

    # XVA Breakdown Chart
    st.subheader("XVA Component Breakdown")

    fig_breakdown = go.Figure(data=[
        go.Bar(
            x=['FVA', 'MVA', 'KVA'],
            y=[xva_results['FVA'], xva_results['MVA'], xva_results['KVA
                ']],
            marker_color=['#6865F2', '#5DFFBC', '#FF6B6B'],
            text=[f"  {x:.2f}" for x in [xva_results['FVA'],
                xva_results['MVA'], xva_results['KVA']]],
            textposition='outside'
        )
    ])

    fig_breakdown.update_layout(
        title="XVA Components",
        xaxis_title="Component",
        yaxis_title="Amount (  )",
        height=400,
        showlegend=False
    )

    st.plotly_chart(fig_breakdown, use_container_width=True)

    # Exposure Profile - Optimized for Performance
    st.subheader("Exposure Profile Analysis")

    if xva_results['FVA_details'] is not None:
        time_grid = xva_results['FVA_details']['time_grid']
        epe = xva_results['FVA_details']['epe']
        exposures = xva_results['FVA_details']['exposures']

        # Static exposure profile - optimized for performance
        fig_exposure = go.Figure()

        if exposures is not None:
            # Calculate confidence bands instead of showing many paths
            # This is much more performant
            percentile_5 = np.percentile(exposures, 5, axis=0)
            percentile_95 = np.percentile(exposures, 95, axis=0)

            # Add confidence band (filled area)
            fig_exposure.add_trace(go.Scatter(
                x=np.concatenate([time_grid, time_grid[::-1]]),
                y=np.concatenate([percentile_5, percentile_95[::-1]]),
                fill='toself',
                fillcolor='rgba(104, 101, 242, 0.1)',
                line=dict(color='rgba(255,255,255,0)'),
                hoverinfo="skip",
                showlegend=False,
```

```
                name='5th-95th Percentile'
            ))

            # Show just a few sample paths for context
            sample_size = min(5, exposures.shape[0])  # Only 5 sample paths
            if sample_size > 0:
                sample_indices = np.random.choice(exposures.shape[0], sample_size, replace=False)

                for idx in sample_indices:
                    fig_exposure.add_trace(go.Scatter(
                        x=time_grid,
                        y=exposures[idx, :],
                        mode='lines',
                        line=dict(color='rgba(104, 101, 242, 0.2)', width=1),
                        showlegend=False,
                        hoverinfo='skip'
                    ))

        # Add EPE (main line)
        fig_exposure.add_trace(go.Scatter(
            x=time_grid,
            y=epe,
            mode='lines',
            name='Expected Positive Exposure',
            line=dict(color='#6865F2', width=3),
            hovertemplate='Time: %{x:.2f} years<br>EPE:   %{y:,.2f}<extra></extra>'
        ))

        # Add zero line
        fig_exposure.add_hline(
            y=0, line_dash="dash", line_color="gray", opacity=0.5
        )

        fig_exposure.update_layout(
            title="Exposure Profile",
            xaxis_title="Time (Years)",
            yaxis_title="Exposure (  )",
            height=500,
            hovermode='x unified'  # Better hover performance
        )

        # Make plot non-interactive for better performance
        st.plotly_chart(
            fig_exposure,
            use_container_width=True,
            config={
                'staticPlot': True,  # Disable interactivity
                'displayModeBar': False  # Hide toolbar
            }
        )

    # Initial Margin Profile
    st.subheader("Initial Margin Profile (MVA)")

```

```
    if xva_results['MVA_details'] is not None:
        im_profile = xva_results['MVA_details']['im_profile']
        time_grid_im = xva_results['MVA_details']['time_grid']

        fig_im = go.Figure()
        fig_im.add_trace(go.Scatter(
            x=time_grid_im,
            y=im_profile,
            mode='lines+markers',
            name='Initial Margin',
            line=dict(color='#5DFFBC', width=3),
            marker=dict(size=6),
            fill='tozeroy',
            fillcolor='rgba(93, 255, 188, 0.2)',
            hovertemplate='Time: %{x:.2f} years<br>IM:   %{y:,.2f}<
                extra></extra>'
        ))

        fig_im.update_layout(
            title="Initial Margin Over Time",
            xaxis_title="Time (Years)",
            yaxis_title="Initial Margin (  )",
            height=400
        )

        st.plotly_chart(fig_im, use_container_width=True)

    # Capital Profile
    st.subheader("Regulatory Capital Profile (KVA)")

    if xva_results['KVA_details'] is not None:
        capital_profile = xva_results['KVA_details']['capital_profile']
        time_grid_kva = xva_results['KVA_details']['time_grid']

        fig_capital = go.Figure()
        fig_capital.add_trace(go.Scatter(
            x=time_grid_kva,
            y=capital_profile,
            mode='lines+markers',
            name='CVA Capital',
            line=dict(color='#FF6B6B', width=3),
            marker=dict(size=6),
            fill='tozeroy',
            fillcolor='rgba(255, 107, 107, 0.2)',
            hovertemplate='Time: %{x:.2f} years<br>Capital:   %{y:,.2f
                }<extra></extra>'
        ))

        fig_capital.update_layout(
            title="Regulatory Capital Requirement Over Time",
            xaxis_title="Time (Years)",
            yaxis_title="Capital Requirement (  )",
            height=400
        )

        st.plotly_chart(fig_capital, use_container_width=True)

    # Portfolio Statistics
```

```
    st.markdown("---")
    st.subheader("Portfolio Statistics")

    col1, col2 = st.columns(2)

    with col1:
        # Portfolio composition by counterparty
        cp_counts = pd.Series([t.counterparty for t in portfolio]).
            value_counts()

        fig_cp = go.Figure(data=[
            go.Bar(
                x=cp_counts.index,
                y=cp_counts.values,
                marker_color='#6865F2',
                text=cp_counts.values,
                textposition='outside'
            )
        ])

        fig_cp.update_layout(
            title="Trades by Counterparty",
            xaxis_title="Counterparty",
            yaxis_title="Number of Trades",
            height=300
        )

        st.plotly_chart(fig_cp, use_container_width=True)

    with col2:
        # Portfolio composition by currency
        currency_counts = pd.Series([t.currency for t in portfolio]).
            value_counts()

        fig_currency = go.Figure(data=[
            go.Pie(
                labels=currency_counts.index,
                values=currency_counts.values,
                hole=0.4,
                marker_colors=['#6865F2', '#5DFFBC', '#FF6B6B']
            )
        ])

        fig_currency.update_layout(
            title="Portfolio by Currency",
            height=300
        )

        st.plotly_chart(fig_currency, use_container_width=True)

    # Detailed Results Table
    st.markdown("---")
    st.subheader("Detailed Results")

    results_df = pd.DataFrame({
        'Component': ['FVA', 'MVA', 'KVA', 'Total XVA'],
        'Amount (  )': [
            xva_results['FVA'],
```

```
            xva_results['MVA'],
            xva_results['KVA'],
            xva_results['Total XVA']
        ],
        'Percentage of Total': [
            xva_results['FVA'] / xva_results['Total XVA'] * 100,
            xva_results['MVA'] / xva_results['Total XVA'] * 100,
            xva_results['KVA'] / xva_results['Total XVA'] * 100,
            100.0
        ]
    })

    st.dataframe(
        results_df.style.format({
            'Amount (  )': '{:,.2f}',
            'Percentage of Total': '{:.2f}%'
        }),
        use_container_width=True,
        hide_index=True
    )

    # Download button
    csv = results_df.to_csv(index=False)
    st.download_button(
        label="Download Results as CSV",
        data=csv,
        file_name="xva_results.csv",
        mime="text/csv"
    )
```

This dashboard provides interactive analysis of XVA components with real-time parameter sensitivity analysis.

Expected Outputs

- Comprehensive XVA calculations (FVA, MVA, KVA) for centrally cleared portfolios.
- Interactive visualisations showing XVA components, exposure profiles, and capital requirements.
- Real-time sensitivity analysis demonstrating how XVA responds to changes in funding spreads, regulatory parameters, and portfolio composition.
- Monte Carlo simulation results for exposure and COA analysis.
- Portfolio-level aggregation with netting and diversification benefits.
- CCP default waterfall analysis and stress testing capabilities.
- Professional dashboard interface suitable for risk management and regulatory reporting.

Interview Questions

1. How does central clearing affect XVA calculations compared to bilateral trading, and what are the key differences in margin requirements?
2. Explain the relationship between funding spreads, initial margin, and regulatory capital in determining total XVA. How would you optimise portfolio composition to minimise XVA?

3. How does the Margin Period of Risk impact Close-Out Amount calculations, and what factors influence the MPoR for CCP-cleared versus bilateral trades?

4. Discuss the challenges of aggregating XVA across multiple counterparties and asset classes. How do netting agreements and correlation assumptions affect portfolio-level KVA?

5. How would you extend this framework to include Dynamic Initial Margin (DIM) and incorporate stochastic funding spread models?

6. What validation and backtesting approaches would you use to ensure XVA calculations are accurate and consistent with market observations?

Resume Bullets

- Developed a comprehensive XVA framework implementing FVA, MVA, and KVA calculations for centrally cleared derivative portfolios using object-oriented Python design.
- Built an interactive Streamlit dashboard with real-time Monte Carlo simulations, exposure analysis, and regulatory capital calculations for post-crisis risk management.
- Implemented ISDA SIMM-based initial margin calculations and CCP default waterfall analysis, incorporating Basel III/IV capital requirements and funding spread dynamics.
- Designed extensible software architecture enabling integration of additional XVA components, alternative margin methodologies, and stress testing capabilities.